

FutureKawa — Architecture globale

Dossier technique — Architecture, choix technologiques et robustesse

Architecture globale

Vue d'ensemble

La solution est découpée en microservices conteneurisés, organisés autour de deux niveaux : un niveau local par exploitation (Brésil / Équateur, dans le prototype : exploitation 1 et exploitation 2) et un niveau central au siège qui consolide les données remontées par les exploitations. Chaque service tourne dans son propre conteneur Docker, ce qui permet de faire évoluer, déployer et redémarrer chaque brique indépendamment.

Composants par exploitation

Pour chaque exploitation (répété à l'identique pour exploitation 1 et exploitation 2), la chaîne est composée des microservices suivants :

- **IoT / simulateur** — un microservice IoT (microcontrôleur réel relié en Arduino) et un microservice simulateur, qui reproduit le comportement d'un capteur température / humidité pour fiabiliser les démonstrations et les tests sans dépendre du matériel physique.
- **Broker MQTT** — un microservice dédié (Mosquitto) qui reçoit les publications du capteur ou du simulateur sur des topics dédiés à l'exploitation.
- **API exploitation** — un microservice Django REST Framework qui s'abonne au broker, persiste les relevés, expose les lots et les mesures, et applique les règles d'alerte (seuils température/humidité, ancienneté des lots).
- **Base de données** — un microservice PostgreSQL dédié à l'exploitation, isolé des autres bases.
- **Front exploitation** — un microservice Vue.js / TypeScript dédié, consommé par les équipes terrain de l'entrepôt (consultation des lots, courbes, alertes).

Composants au siège

Le siège dispose de sa propre chaîne, structurée de la même façon que les exploitations mais orientée consolidation :

- **API pays / API siège** — un microservice qui interroge les API exploitation via leurs routes REST exposées, agrège l'état des stocks, les mesures et les alertes des différentes exploitations.
- **Base de données siège** — un microservice PostgreSQL dédié, qui stocke la vue consolidée et les sauvegardes utilisées en cas d'indisponibilité d'une exploitation.
- **Front siège** — un microservice Vue.js / TypeScript distinct des fronts exploitation, destiné à la supervision multi-sites.

Composants transverses

- **Déploiement** — un microservice dédié au déploiement (orchestration des montées de version, scripts de mise en production).
- **Infra** — un microservice portant la définition d'infrastructure (Terraform).
- **Documentation** — un microservice dédié à la génération et à l'hébergement de la documentation technique et utilisateur du projet.

Flux de données

- Capteur / simulateur → Broker MQTT (publication des relevés température/humidité par topic exploitation).
- Broker MQTT → API exploitation (abonnement, validation, application des seuils, persistance en base).
- Front exploitation → API exploitation (consultation des lots, courbes, alertes en HTTP/REST).
- API siège → API exploitation (interrogation périodique des routes REST exposées par chaque exploitation pour consolidation).
- Front siège → API siège (vue consolidée multi-exploitations).

Synthèse des microservices

Microservice	Niveau	Rôle
Front exploitation 1 / 2	Local	Interface terrain entrepôt
Front siège	Central	Supervision multi-sites
API exploitation	Local	Gestion lots, mesures, alertes
API pays / siège	Central	Consolidation multi-exploitations
BDD exploitation 1 / 2	Local	Persistance PostgreSQL locale
BDD siège	Central	Persistance consolidée + backups
IoT (Arduino)	Local	Capteur température/humidité réel
Simulateur	Local	Génération de relevés simulés
Broker MQTT (Mosquitto)	Local	Transport des relevés capteurs
Infra (Terraform)	Transverse	Provisioning de l'infrastructure
Déploiement	Transverse	Orchestration des mises en production
Documentation	Transverse	Documentation technique/utilisateur

Justification des choix technologiques et du découpage

Découpage en microservices

Le découpage suit la structure métier de FutureKawa : une exploitation = une chaîne autonome (capteur, broker, API, base, front), et un siège = un niveau de consolidation. Ce choix reflète directement l'architecture imposée par le cahier des charges (un backend local par pays + un backend central au siège) et apporte trois bénéfices :

- **Isolation des pannes** — une exploitation qui tombe n'affecte ni les autres exploitations ni le siège, qui continue de fonctionner avec les dernières données connues.
- **Déploiement indépendant** — chaque service peut être mis à jour, redémarré ou redimensionné sans interrompre les autres.
- **Clarté de responsabilité** — chaque conteneur a un rôle unique et bien identifié, ce qui simplifie les tests, les logs et la maintenance.

La séparation explicite entre IoT réel et simulateur permet de développer et démontrer la chaîne complète indépendamment de la disponibilité du matériel physique, et de produire des jeux de données reproductibles pour les tests.

Stack technique

Brique	Choix et justification
Conteneurisation	Docker / Docker Compose, images Alpine : empreinte réduite, démarrage rapide, cohérent avec l'attendu « docker compose up » du cahier des charges.
Backend / API	Django REST Framework (Python) : framework mature, ORM intégré pour Postgres, sérialisation REST native, écosystème adapté à un découpage en plusieurs API (exploitation, siège).
Base de données	PostgreSQL : fiabilité, support transactionnel solide, adapté au stockage de séries temporelles (relevés) et aux relations lots/exploitations/mesures.
Frontend	Vue.js + TypeScript : typage statique pour fiabiliser les échanges avec les API, framework léger adapté à des interfaces distinctes (front exploitation vs front siège).
IoT	Arduino : prise en main rapide, large documentation, suffisant pour un prototype de relevé température/humidité.
Broker	Mosquitto : implémentation MQTT légère et standard, déployable en conteneur, alignée avec les ressources fournies dans le cahier des charges.
Infrastructure	Terraform : description déclarative et versionnée de l'infrastructure, reproductible entre exploitations.
CI/CD	GitLab CI/CD : build, tests et packaging automatisés à chaque commit, intégré nativement au dépôt Git du projet.
Lancement local	Makefile : point d'entrée unique pour orchestrer les commandes Docker Compose, simplifiant la prise en main par un nouvel arrivant ou le jury.

Éléments de robustesse

Tolérance aux pannes du module IoT

- En cas de perte de connexion ou d'arrêt du capteur, un message d'erreur est remonté et affiché côté front, afin que l'utilisateur terrain identifie immédiatement l'incident plutôt que de voir des données figées sans explication.
- Une marge d'erreur est appliquée sur les relevés du capteur : si aucune valeur n'est reçue dans l'intervalle attendu, le système le détecte et le signale au lieu d'afficher une valeur non fiable ou obsolète sans avertissement.

Isolation entre exploitations

- Chaque front exploitation est un microservice indépendant des autres : l'indisponibilité du front (ou de la chaîne) de l'exploitation 1 n'a aucun impact sur le fonctionnement de l'exploitation 2.
- Cette isolation découle directement du découpage en microservices avec une base de données et une API dédiées par exploitation, sans dépendance directe entre exploitations.

Continuité de service au siège

- Si le siège ne parvient plus à récupérer les informations d'une exploitation (panne réseau, service indisponible), il affiche automatiquement la dernière sauvegarde connue, avec une indication explicite de la date/heure des données affichées, pour que l'utilisateur sache qu'il consulte un état antérieur et non une donnée temps réel.

- Ce mécanisme garantit que le siège reste exploitable en consultation même en cas de panne partielle, sans bloquer la supervision globale.

Logs et supervision

- Chaque microservice écrit ses logs sur la sortie standard (stdout), collectée par Docker, ce qui permet de consulter l'activité et les erreurs de chaque conteneur indépendamment via les commandes Docker standard (ex : docker compose logs).
- Ce niveau reste volontairement simple pour le prototype ; une supervision centralisée (agrégation des logs, métriques, alerting infra) constitue une évolution naturelle pour la phase d'industrialisation.